

Security Audit

Aegis (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Aegis team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Aegis is a decentralized finance (DeFi) protocol built on Ethereum that offers a Bitcoin-backed stablecoin called YUSD. It utilizes a delta-neutral strategy—holding BTC collateral while hedging with perpetual futures—to maintain price stability and generate yield from funding fees. The platform emphasizes full transparency, with real-time proof-of-reserves and an intuitive dashboard, and is governed by a DAO with its own governance token (AEG), enabling community-driven protocol decisions.

Project Name: Aegis

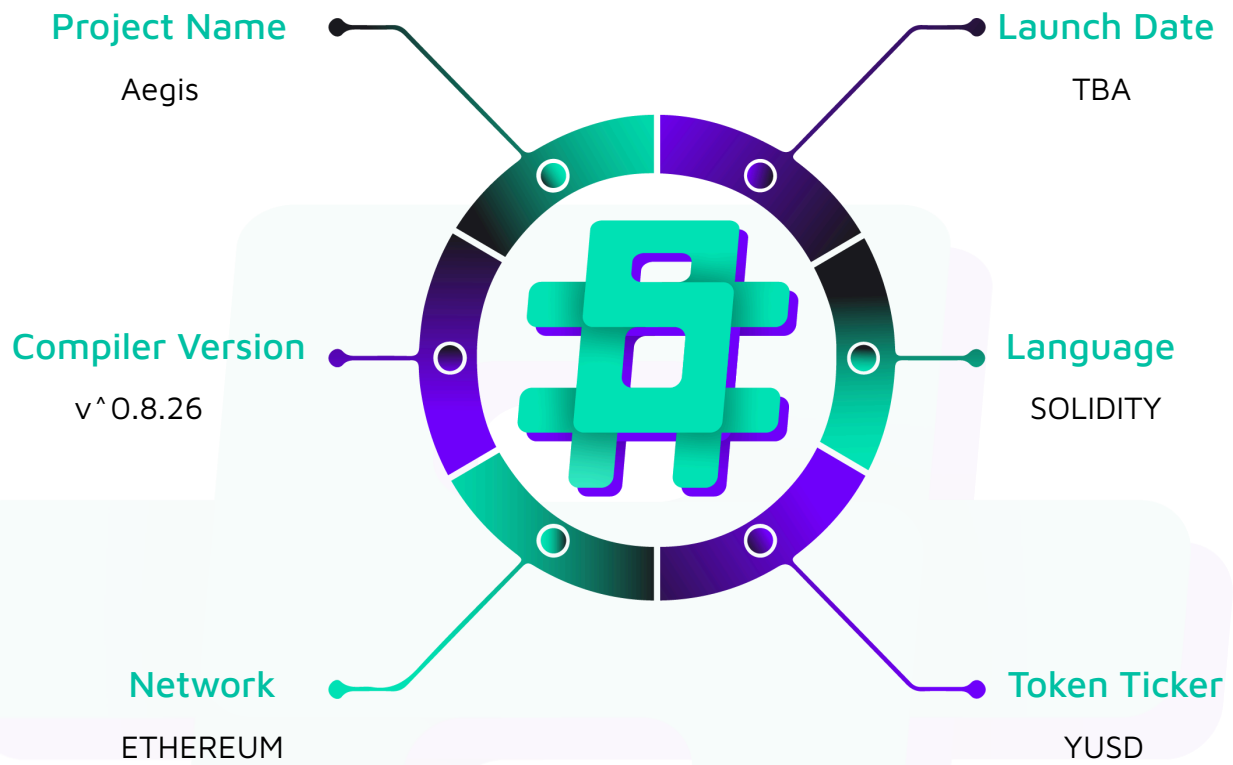
Project Type: DeFi

Compiler Version: ^0.8.26

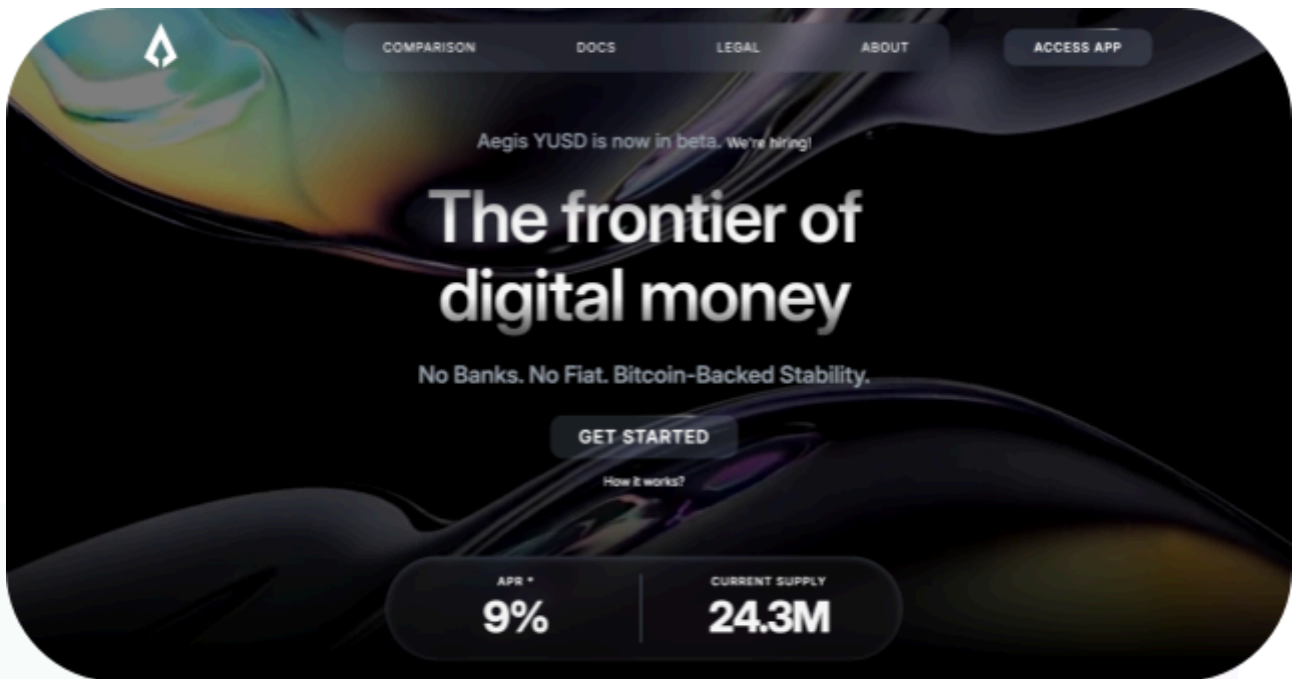
Website: <https://aegis.im/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Aegis project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Aegis Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract 1	sYUSD.sol
Contract 2	sYUSDSilo.sol
Audited GitHub Commit Hash	7c84d5111ff228dec378a798d2d8ce8813a3ece0
Fix Review GitHub Commit Hash	24caf1f72a48455c10071f98cde41b65ddaa8692

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

- 1 High severity vulnerability
- 2 Low severity vulnerabilities
- 2 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
sYUSD.sol - Deposit YUSD to receive sYUSD shares - Withdraw YUSD directly (if cooldown is off) - Redeem sYUSD shares for YUSD directly (if cooldown is off) - Initiate cooldown for assets to unstake (if cooldown is on) - Initiate cooldown for shares to unstake (if cooldown is on) - Unstake YUSD after cooldown period - View user cooldown status - Set cooldown duration - Rescue non-YUSD ERC20 tokens accidentally sent to the contract	Contract achieves this functionality.
sYUSDSilo.sol - View YUSD token address - View staking vault address - Withdraw YUSD to a specified address	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Aegis project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Aegis project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] sYUSD#cooldownShares&cooldownAssets - Unauthorized CoolDown Manipulation Leads to Griefing Attacks

Description

In the `cooldownAssets` and `cooldownShares` functions, any user can initiate a cooldown for another user's assets by specifying the owner parameter. This is problematic because a malicious actor could repeatedly call these functions for other users, locking their assets in a cooldown state and preventing them from unstaking until the cooldown period expires.

Vulnerability Details

In `cooldownAssets` and `cooldownShares`, the line `cooldowns[owner].cooldownEnd = uint104(block.timestamp) + cooldownDuration;` is executed regardless of the current state of `cooldowns[owner]`. An attacker can call `cooldownAssets(0, victim_address)` or `cooldownShares(0, victim_address)`.

- If assets or shares are 0, the subsequent `super.withdraw` or `super.redeem` calls will attempt to process 0 shares/assets. This means they will not require any sYUSD allowance from the `victim_address` (as `allowance(victim, attacker) < 0` is false). The `_burn` and `_transfer` operations inside ERC4626 will also be no-ops for 0 amounts.
- However, the `cooldownEnd` for `victim_address` will be reset to `block.timestamp + cooldownDuration`, effectively extending their wait time indefinitely if the attacker calls this repeatedly just before the cooldown is about to end.

Impact

A malicious actor can prevent or indefinitely delay any user from unstaking their assets. And users may be locked out of their funds indefinitely through griefing attacks.

Recommendation

Generally, it is recommended to revert if assets/shares is 0 and the `cooldownEnd` should be only set if the user is starting a new cooldown cycle and also make sure, the owner is `msg.sender`.

```
if (assets == 0) revert ZeroAmount(); // In cooldownAssets
if (shares == 0) revert ZeroAmount(); // In cooldownShares

(OR)

// In cooldownAssets (and similarly in cooldownShares)
Cooldown storage userCooldown = cooldowns[owner];
if (userCooldown.underlyingAmount == 0) { // Only set/reset cooldownEnd if starting a new batch
    userCooldown.cooldownEnd = uint104(block.timestamp) + cooldownDuration;
}
// ... rest of the logic ...
userCooldown.underlyingAmount += uint152(assets); // Or 'assets' from redeem for cooldownShares
```

Status

Resolved

Low

[L-01] sYUSD - Unintended Admin Role Assignment might leads to losing access over contracts

Description

The contract assigns admin roles to a parameter `admin` instead of `msg.sender` during initialization. While not necessarily a vulnerability, this pattern can sometimes leads to deployment issues as the admin might be a different address from expected.

Recommendation

Consider assigning the admin role to `msg.sender(the deployer)` by default, or implement additional validation.

Status

Acknowledged

[L-02] sYUSD - Missing Zero-Address Checks

Description

Several functions lack zero-address checks for critical parameters.

- `unstake(address receiver)`: receiver is not checked for `address(0)`. Sending assets to `address(0)` would burn them.
- `rescueTokens(address token, uint256 amount, address to)`: to and token are not checked for `address(0)`. While `token == asset()` is checked, token itself could be `address(0)`. The amount is also not checked for 0.

Recommendation

Add zero-address/zero-amount checks

Status

Resolved

QA

[Q-01] sYUSD - Consider Events for Admin Actions

Description

`rescueTokens` performs a critical action. While an event is emitted by `safeTransfer`, an explicit event like `TokensRescued`(address indexed token, address indexed to, `uint256` amount) could improve traceability specific to this contract's operation. (Not a bug, but good practice).

Recommendation

Implement `TokensRescue` event

Status

Resolved

[Q-02] sYUSD - `ADMIN_ROLE` vs `DEFAULT_ADMIN_ROLE`

Description

The initializer grants both roles to the admin address. This implies `DEFAULT_ADMIN_ROLE` is the higher privilege (as it typically can manage other roles). This separation is fine, just important to note for role management. If `DEFAULT_ADMIN_ROLE` is renounced by the sole holder, `ADMIN_ROLE` might become unmanageable if no one else has `DEFAULT_ADMIN_ROLE`. Standard pattern, but requires careful admin key management.

Recommendation

Just assign a `DEFAULT_ADMIN_ROLE`

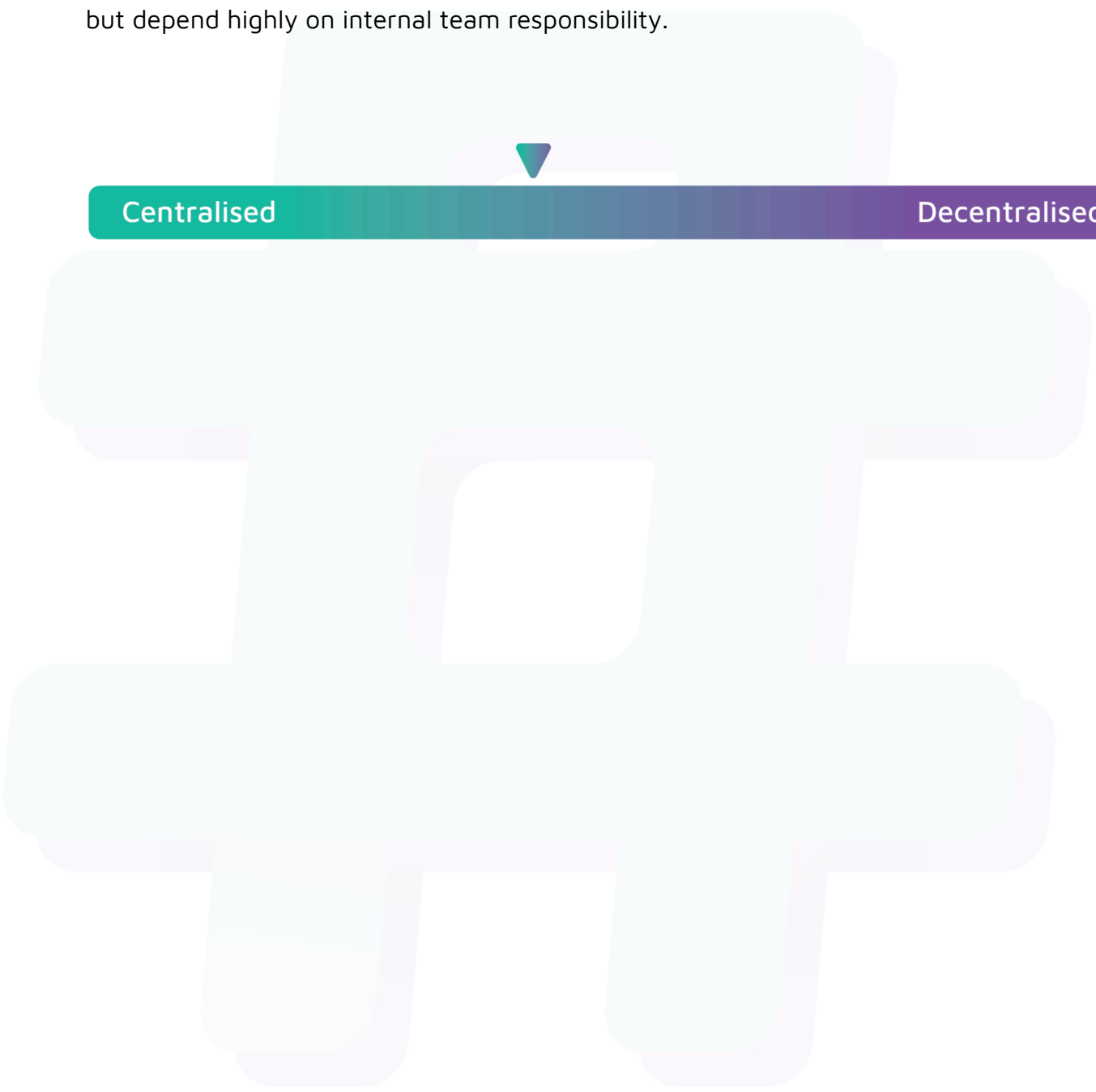
Status

Resolved

Centralisation

The Aegis project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Aegis project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.